<u>Machine Learning & Deep Learning</u>

Experiment 4

Name: Abhishek Vishwakarma
Class: D15C
Roll No: 47

AIM: Implement K-Nearest Neighbors (KNN) and evaluate model performance.

1. Dataset Source

- **Dataset Name:** Social Network Ads
- **Source:** [Kaggle - Social Network Ads]

2. Dataset Description

This dataset contains information about users on a social network, including their gender, age, and estimated salary. The goal is to predict whether a user will purchase a specific product (e.g., a car or insurance) based on these demographics.

- **Size:** 400 records, 5 columns.
- **Target Variable:** Purchased (0 = No, 1 = Yes).
- **Features:**
    - **User ID**: Unique ID (irrelevant for modeling).
    - **Gender**: Male/Female.
    - **Age**: Age of the user.
    - **EstimatedSalary**: User's estimated annual salary.

3. Mathematical Formulation of the Algorithm

KNN is a **Lazy Learner**. It does not "learn" a model (like finding coefficients in regression). Instead, it memorizes the training dataset.

Prediction Step:

When a new data point xnew arrives:

1. Calculate the distance between xnew and every point in the training set.
2. Select the K nearest neighbors (data points with the smallest distances).
3. **Majority Vote:** Assign the class that is most frequent among these K neighbors.

Distance Metrics:

The most common metric is **Euclidean Distance**:

$$d(p, q) = \sqrt{\sum_{i=1}^{n} (q_i - p_i)^2}$$

Where p and q are two data points in n-dimensional space.

4. Algorithm Limitations

- **Computationally Expensive:** Since it calculates the distance to *every* training point for *every* prediction, it becomes very slow with large datasets.
- **Sensitive to Outliers:** A single outlier can dramatically shift the prediction if K is small.
- **Curse of Dimensionality:** In high-dimensional spaces (many features), the concept of "distance" becomes less meaningful, and KNN performance degrades.
- **Feature Scaling is Mandatory:** If one feature is "Salary" (range 20,000-150,000) and another is "Age" (range 20-60), the distance calculation will be completely dominated by Salary.

5. Methodology / Workflow

1. **Data Loading:** Load Social_Network_Ads.csv.

```
# 1. Load Dataset
try:
    df = pd.read_csv('Social_Network_Ads.csv')
    print("Dataset loaded successfully.")
except FileNotFoundError:
    print("Error: 'Social_Network_Ads.csv' not found. Please upload the file.")
    df = pd.DataFrame()
```

2. **Preprocessing:**
    - Drop User ID and Gender (to simplify visualization to 2D: Age vs Salary).
    - **Feature Scaling:** Apply StandardScaler. This is the most critical step for KNN.

```
# 2. Preprocessing
# We will use 'Age' and 'EstimatedSalary' to predict 'Purchased'
# Dropping Gender for simpler 2D distance calculation demonstration
X = df[['Age', 'EstimatedSalary']]
y = df['Purchased']
```

3. **Splitting:** Train (75%) / Test (25%).

```
# 3. Split Data
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.25, random_state=42)
```

4. **Model Training:** Instantiate KNeighborsClassifier.
5. **Hyperparameter Tuning:** Use the "Elbow Method" to find the optimal K value (balancing overfitting vs underfitting).

```
# ========================================
# Choosing the Best K (Elbow Method)
# ========================================
error_rate = []

# Will take some time
for i in range(1, 40):
    knn = KNeighborsClassifier(n_neighbors=i)
    knn.fit(X_train, y_train)
    pred_i = knn.predict(X_test)
    # Error rate = Mean of where predictions did not match actual
    error_rate.append(np.mean(pred_i != y_test))

# Plotting Error Rate vs K Value
plt.figure(figsize=(10, 6))
plt.plot(range(1, 40), error_rate, color='blue', linestyle='dashed', marker='o',
         markerfacecolor='red', markersize=10)
plt.title('Error Rate vs. K Value')
plt.xlabel('K')
plt.ylabel('Error Rate')
plt.grid(True)
plt.show()
```

6. **Evaluation:** Accuracy Score and Confusion Matrix.

```
# ========================================
# Evaluation
# ========================================
acc = accuracy_score(y_test, y_pred)
print(f"KNN Accuracy: {acc:.4f}")

print("\nClassification Report:")
print(classification_report(y_test, y_pred))

# Confusion Matrix Visualization
plt.figure(figsize=(6, 5))
cm = confusion_matrix(y_test, y_pred)
sns.heatmap(cm, annot=True, fmt='d', cmap='Greens',
            xticklabels=['Not Purchased', 'Purchased'],
            yticklabels=['Not Purchased', 'Purchased'])
plt.xlabel('Predicted')
plt.ylabel('Actual')
plt.title(f'Confusion Matrix (K={k_optimal})')
plt.show()
```

Code:

```
# ==============================================================
# EXPERIMENT 4: K-NEAREST NEIGHBORS (KNN)
# ==============================================================

import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import accuracy_score, classification_report, confusion_matrix

print("--- EXPERIMENT 4: K-NEAREST NEIGHBORS (KNN) ---")

# 1. Load Dataset
try:
    df = pd.read_csv('Social_Network_Ads.csv')
    print("Dataset loaded successfully.")
except FileNotFoundError:
    print("Error: 'Social_Network_Ads.csv' not found. Please upload the file.")
    df = pd.DataFrame()

if not df.empty:
    # 2. Preprocessing
    # We will use 'Age' and 'EstimatedSalary' to predict 'Purchased'
    # Dropping Gender for simpler 2D distance calculation demonstration
    X = df[['Age', 'EstimatedSalary']]
    y = df['Purchased']

    # 3. Split Data
    X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.25,
random_state=42)

    # 4. Feature Scaling (CRITICAL FOR KNN)
    scaler = StandardScaler()
    X_train = scaler.fit_transform(X_train)
    X_test = scaler.transform(X_test)
```

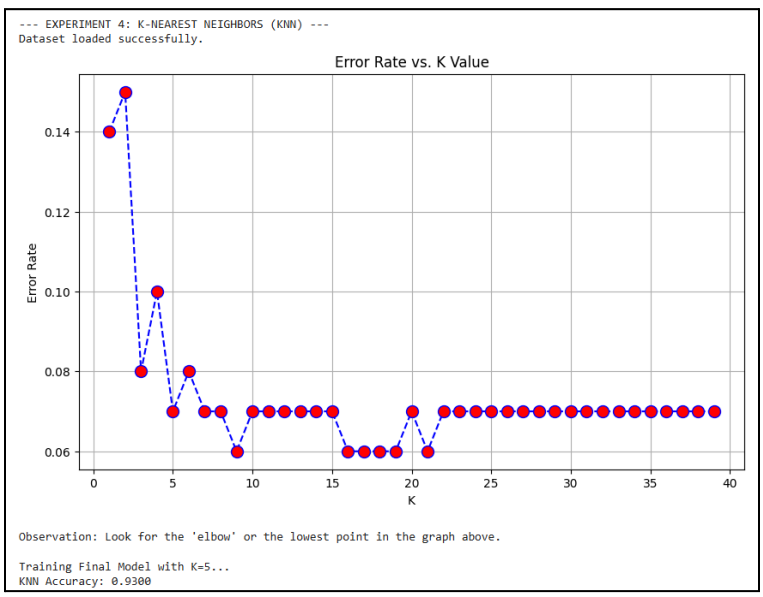
```
    # ═══════════════════════════════════════════
    # Choosing the Best K (Elbow Method)
    # ═══════════════════════════════════════════
    error_rate = []

    # Will take some time
    for i in range(1, 40):
        knn = KNeighborsClassifier(n_neighbors=i)
        knn.fit(X_train, y_train)
        pred_i = knn.predict(X_test)
        # Error rate = Mean of where predictions did not match actual
        error_rate.append(np.mean(pred_i ≠ y_test))

    # Plotting Error Rate vs K Value
    plt.figure(figsize=(10, 6))
    plt.plot(range(1, 40), error_rate, color='blue', linestyle='dashed', marker='o',
             markerfacecolor='red', markersize=10)
    plt.title('Error Rate vs. K Value')
    plt.xlabel('K')
    plt.ylabel('Error Rate')
    plt.grid(True)
    plt.show()

    print("\nObservation: Look for the 'elbow' or the lowest point in the graph
above.")

    # ═══════════════════════════════════════════
    # Train with Optimal K
    # ═══════════════════════════════════════════
    # Let's assume K=5 is a good balance based on standard heuristics
    # (You can adjust this based on the graph generated above)
    k_optimal = 5
    print(f"\nTraining Final Model with K={k_optimal}...")

    classifier = KNeighborsClassifier(n_neighbors=k_optimal, metric='minkowski', p=2)
    classifier.fit(X_train, y_train)

    y_pred = classifier.predict(X_test)
```

```python
# ═══════════════════════════════════════════
# Evaluation
# ═══════════════════════════════════════════
acc = accuracy_score(y_test, y_pred)
print(f"KNN Accuracy: {acc:.4f}")

print("\nClassification Report:")
print(classification_report(y_test, y_pred))

# Confusion Matrix Visualization
plt.figure(figsize=(6, 5))
cm = confusion_matrix(y_test, y_pred)
sns.heatmap(cm, annot=True, fmt='d', cmap='Greens',
            xticklabels=['Not Purchased', 'Purchased'],
            yticklabels=['Not Purchased', 'Purchased'])
plt.xlabel('Predicted')
plt.ylabel('Actual')
plt.title(f'Confusion Matrix (K={k_optimal})')
plt.show()

# ═══════════════════════════════════════════
# Visualizing Decision Boundaries (2D)
# ═══════════════════════════════════════════
# This visualizes how KNN divides the "Age vs Salary" plane
from matplotlib.colors import ListedColormap

X_set, y_set = X_test, y_test
X1, X2 = np.meshgrid(np.arange(start = X_set[:, 0].min() - 1, stop = X_set[:,
0].max() + 1, step = 0.01),
                     np.arange(start = X_set[:, 1].min() - 1, stop = X_set[:,
1].max() + 1, step = 0.01))

plt.figure(figsize=(10,6))
plt.contourf(X1, X2, classifier.predict(np.array([X1.ravel(),
X2.ravel()]).T).reshape(X1.shape),
             alpha = 0.75, cmap = ListedColormap(('red', 'green')))
plt.xlim(X1.min(), X1.max())
plt.ylim(X2.min(), X2.max())
```

```
# Plot data points
# Note: iterating through numpy arrays for scattering
for i, j in enumerate(np.unique(y_set)):
    plt.scatter(X_set[y_set == j, 0], X_set[y_set == j, 1],
                c = ListedColormap(('red', 'green'))(i), label = j)

plt.title(f'KNN Decision Boundary (K={k_optimal})')
plt.xlabel('Age (Scaled)')
plt.ylabel('Estimated Salary (Scaled)')
plt.legend()
plt.show()
```

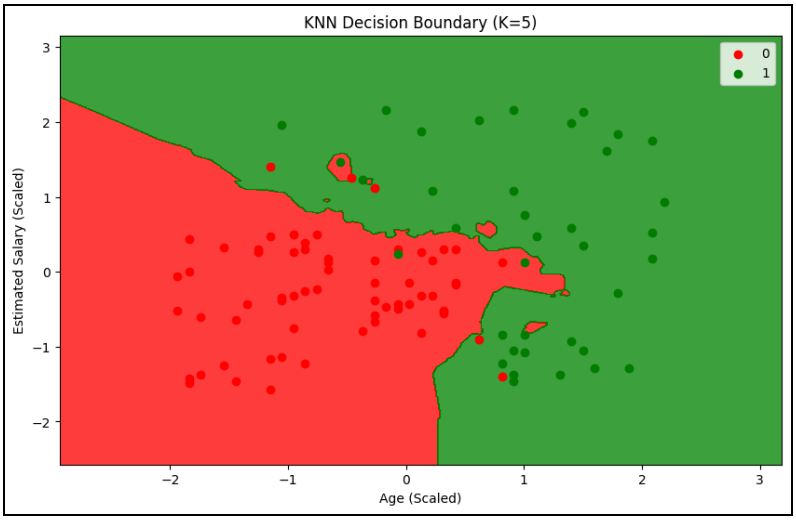
6. Performance Analysis

- **Metric:** Error Rate vs. K-Value.
- **Analysis:**
  - **Small K (e.g., K=1):** The model is too complex, capturing noise (Overfitting). The decision boundary is jagged.
  - **Large K:** The model becomes too simple, voting for the majority class regardless of local nuance (Underfitting). The decision boundary becomes smooth.

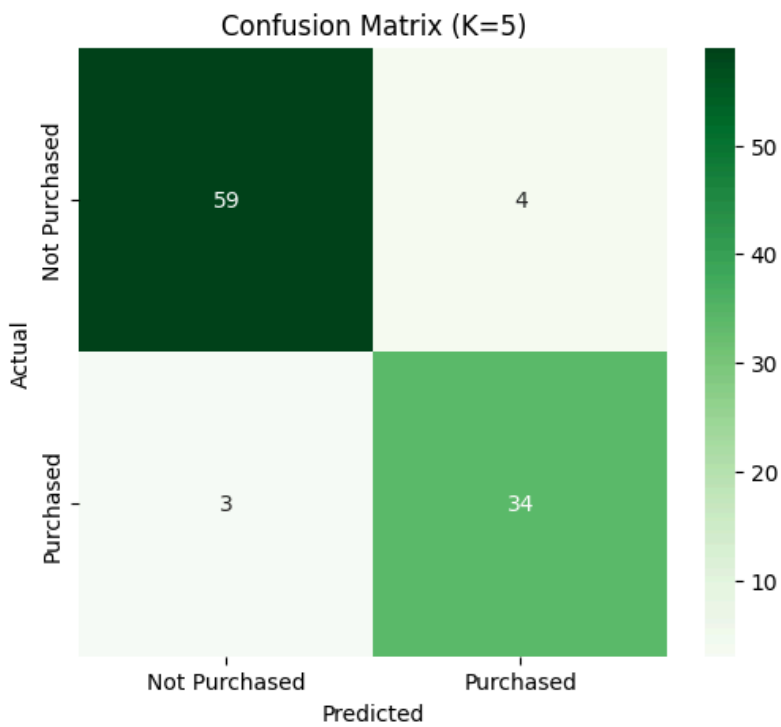


7. Hyperparameter Tuning

- **Parameter:** n_neighbors (K).
- **Process:** We iterate K from 1 to 40 and plot the Error Rate. The optimal K is where the error is low but the model isn't over-smoothing (often around K=5 to 10).




```
Classification Report:
              precision    recall  f1-score   support

           0       0.95      0.94      0.94        63
           1       0.89      0.92      0.91        37

    accuracy                           0.93       100
   macro avg       0.92      0.93      0.93       100
weighted avg       0.93      0.93      0.93       100
```



8. Conclusion

In this experiment, we utilized the K-Nearest Neighbors (KNN) algorithm to predict customer purchasing behavior.

- **Scaling:** We observed that feature scaling (Standardization) was essential because the "Salary" feature had a much larger magnitude than "Age". Without scaling, the distance metric would have been biased entirely toward salary differences.
- **K-Value:** The "Elbow Method" graph demonstrated how the error rate fluctuates with K. A very small K resulted in a highly irregular decision boundary (overfitting), while the optimal K (around 5) produced a generalized boundary that accurately separated high-salary/older users (buyers) from low-salary/younger users (non-buyers).